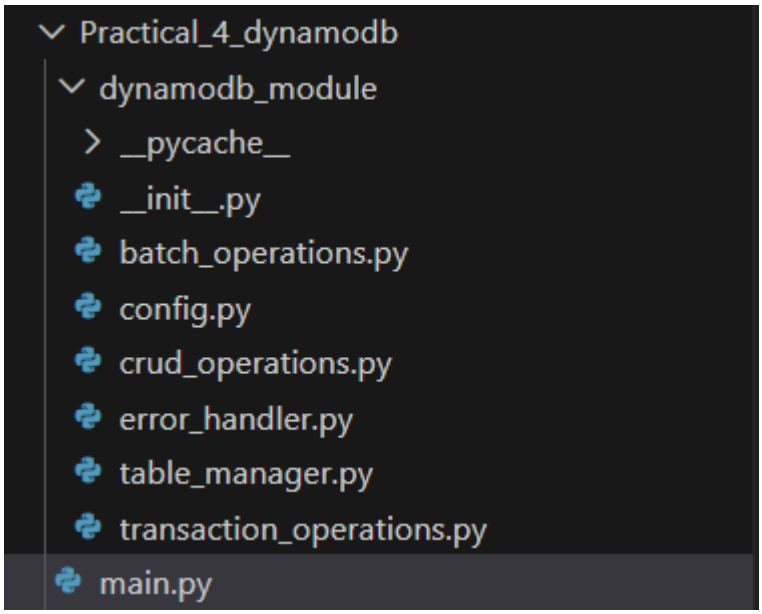
Name : Darshan Ganesh Bhabad

Prn : 202301040169

CAN Practicval No:4

Python module/package (using boto3) that provides robust, production-lean operations with correct key design, indexes, pagination, conditional updates, batch operations, and transactions for DynamoDB.

Structure:



Codes:

Config.py

```
import boto3


def get_dynamodb_resource():
    return boto3.resource('dynamodb', region_name='ap-south-1')


def get_dynamodb_client():
    return boto3.client('dynamodb', region_name='ap-south-1')
```

table_manager.py

```
#Creates DynamoDB table with composite key + GSI.
from .config import get_dynamodb_resource


def create_table():
    dynamodb = get_dynamodb_resource()


    table = dynamodb.create_table(
        TableName='StudentCourses',
        KeySchema=[
            {'AttributeName': 'student_id', 'KeyType': 'HASH'},
            {'AttributeName': 'course_id', 'KeyType': 'RANGE'}
        ],
        AttributeDefinitions=[
            {'AttributeName': 'student_id', 'AttributeType': 'S'},
            {'AttributeName': 'course_id', 'AttributeType': 'S'},
            {'AttributeName': 'status', 'AttributeType': 'S'}
        ],
```

```
    GlobalSecondaryIndexes=[
        {
            'IndexName': 'StatusIndex',
            'KeySchema': [
                {'AttributeName': 'status', 'KeyType': 'HASH'}
            ],
            'Projection': {
                'ProjectionType': 'ALL'
            },
            'ProvisionedThroughput': {
                'ReadCapacityUnits': 5,
                'WriteCapacityUnits': 5
            }
        }
    ],
    ProvisionedThroughput={
        'ReadCapacityUnits': 5,
        'WriteCapacityUnits': 5
    }
)

table.wait_until_exists()
print("Table created successfully")
```

crud_operations.py

```
from .config import get_dynamodb_resource
from boto3.dynamodb.conditions import Key
from botocore.exceptions import ClientError
from .error_handler import handle_error
```

```
dynamodb = get_dynamodb_resource()
table = dynamodb.Table('StudentCourses')

def insert_item(item):
    try:
        table.put_item(Item=item)
        print("Item inserted")

    except ClientError as e:
        handle_error(e)

def query_by_student(student_id):

    response = table.query(
        KeyConditionExpression=Key('student_id').eq(student_id)
    )

    for item in response['Items']:
        print(item)

def query_by_status(status):

    response = table.query(
        IndexName='StatusIndex',
        KeyConditionExpression=Key('status').eq(status)
    )

    for item in response['Items']:
        print(item)
```

```
def update_marks(student_id, course_id, new_marks):

    try:
        table.update_item(
            Key={
                'student_id': student_id,
                'course_id': course_id
            },
            UpdateExpression="SET marks = :m",
            ConditionExpression="marks < :m",
            ExpressionAttributeValues={
                ':m': new_marks
            }
        )

        print("Marks updated")

    except ClientError as e:
        handle_error(e)

def delete_item(student_id, course_id):

    try:
        table.delete_item(
            Key={
                'student_id': student_id,
                'course_id': course_id
            }
        )

        print("Item deleted successfully")
```

```
    except ClientError as e:
        handle_error(e)

def scan_with_pagination():

    response = table.scan()

    while True:

        for item in response['Items']:
            print(item)

        if 'LastEvaluatedKey' not in response:
            break

        response = table.scan(
            ExclusiveStartKey=response['LastEvaluatedKey']
        )
```

Batch_operations.py

```
from .config import get_dynamodb_resource

dynamodb = get_dynamodb_resource()
table = dynamodb.Table('StudentCourses')

def batch_insert(items):

    with table.batch_writer() as batch:

        for item in items:
            batch.put_item(Item=item)
```

```
    print("Batch insert completed")
```

transaction_operations.py

```
from .config import get_dynamodb_client


client = get_dynamodb_client()


def transactional_insert():


    client.transact_write_items(
        TransactItems=[
            {
                'Put': {
                    'TableName': 'StudentCourses',
                    'Item': {
                        'student_id': {'S': 'S500'},
                        'course_id': {'S': 'C900'},
                        'name': {'S': 'TransactionUser'},
                        'status': {'S': 'active'}
                    }
                }
            }
        ]
    )


    print("Transaction completed")
```

error_handler.py

```
#for handling Aws exceptions

from botocore.exceptions import ClientError

import time


def handle_error(e):

    code = e.response['Error']['Code']


    if code == 'ProvisionedThroughputExceededException':

        print("Throughput exceeded. Retrying...")

        time.sleep(2)


    elif code == 'ConditionalCheckFailedException':

        print("Conditional check failed.")


    else:

        print("Unexpected error:", e)
```

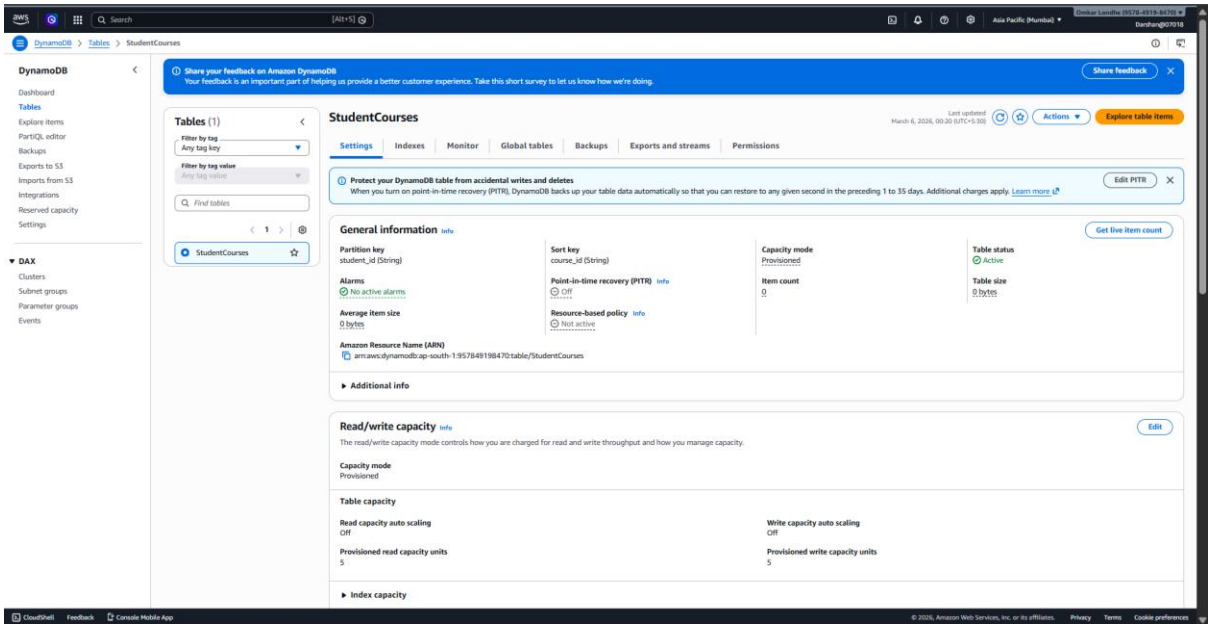
main.py

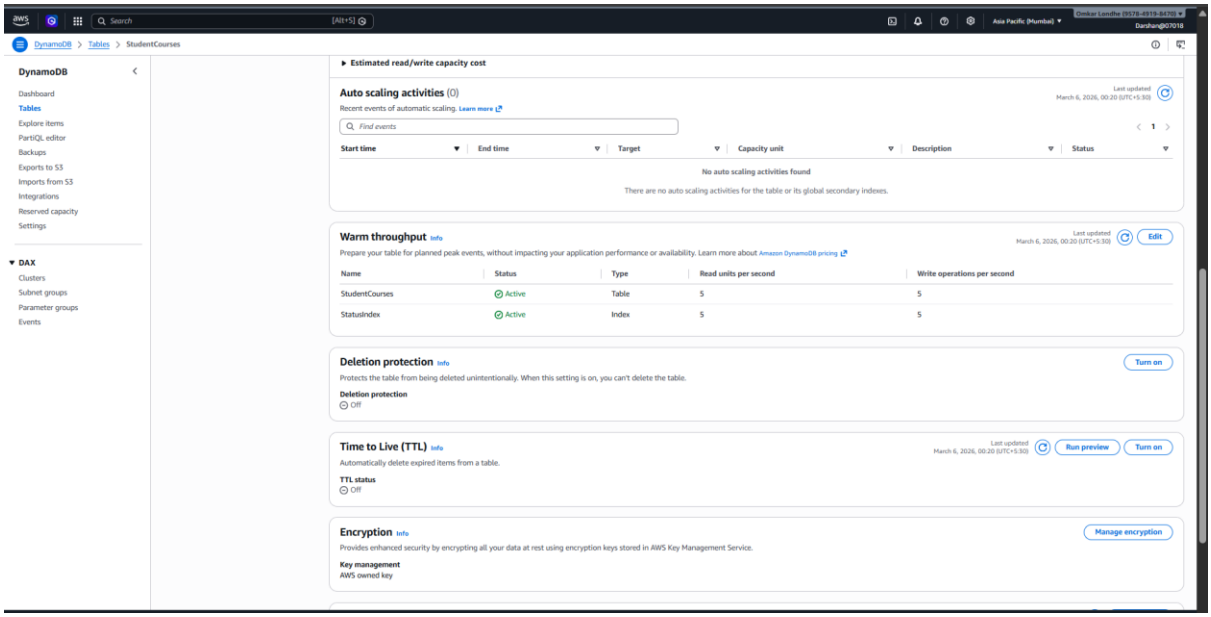
```
from dynamodb_module.table_manager import create_table

from dynamodb_module.crud_operations import insert_item, query_by_student,
update_marks,delete_item

from dynamodb_module.batch_operations import batch_insert

from dynamodb_module.transaction_operations import transactional_insert


create_table()


insert_item({

    "student_id": "S101",

    "course_id": "C001",
```

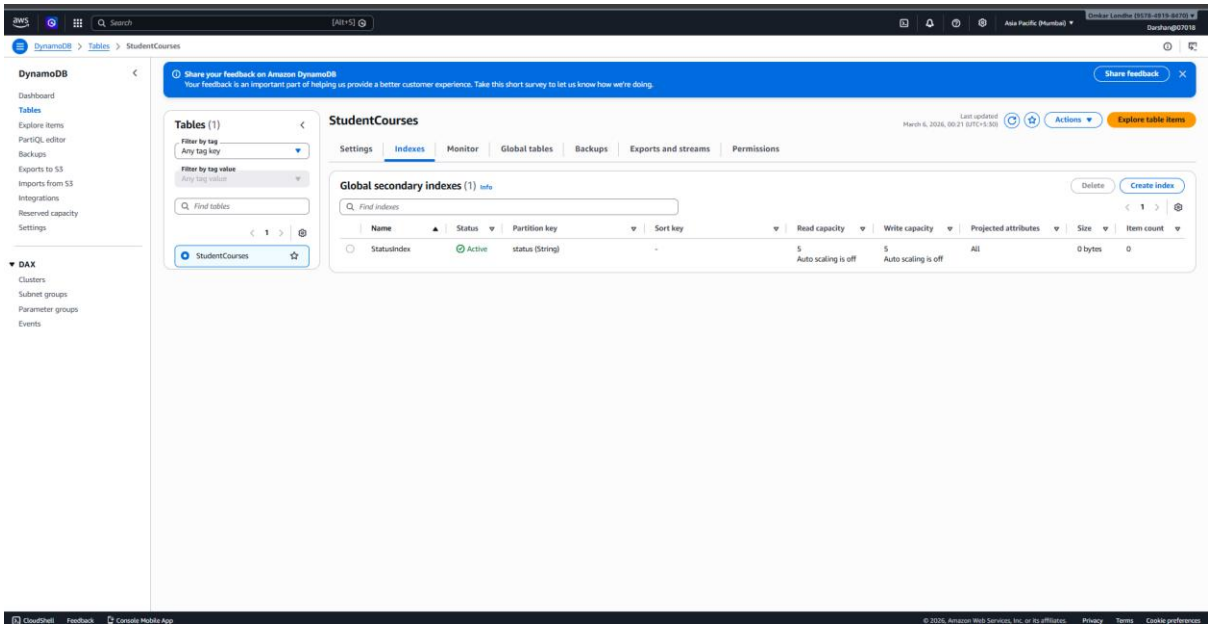
```
    "name": "Darshan",
    "status": "active",
    "marks": 85
})

query_by_student("S101")

update_marks("S101", "C001", 90)

batch_insert([
    {
        "student_id": "S102",
        "course_id": "C002",
        "name": "Virendra",
        "status": "active",
        "marks": 75
    },
    {
        "student_id": "S103",
        "course_id": "C003",
        "name": "Rohan",
        "status": "inactive",
        "marks": 80
    }
])

transactional_insert()
delete_item("S101", "C001")
```

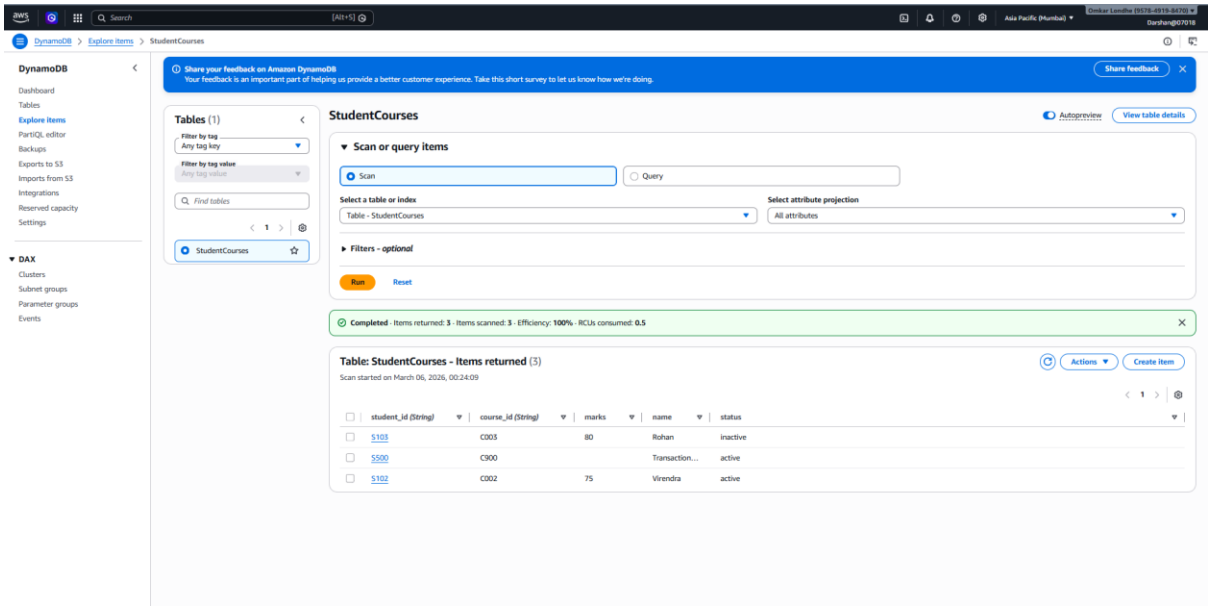
Output Screenshots:

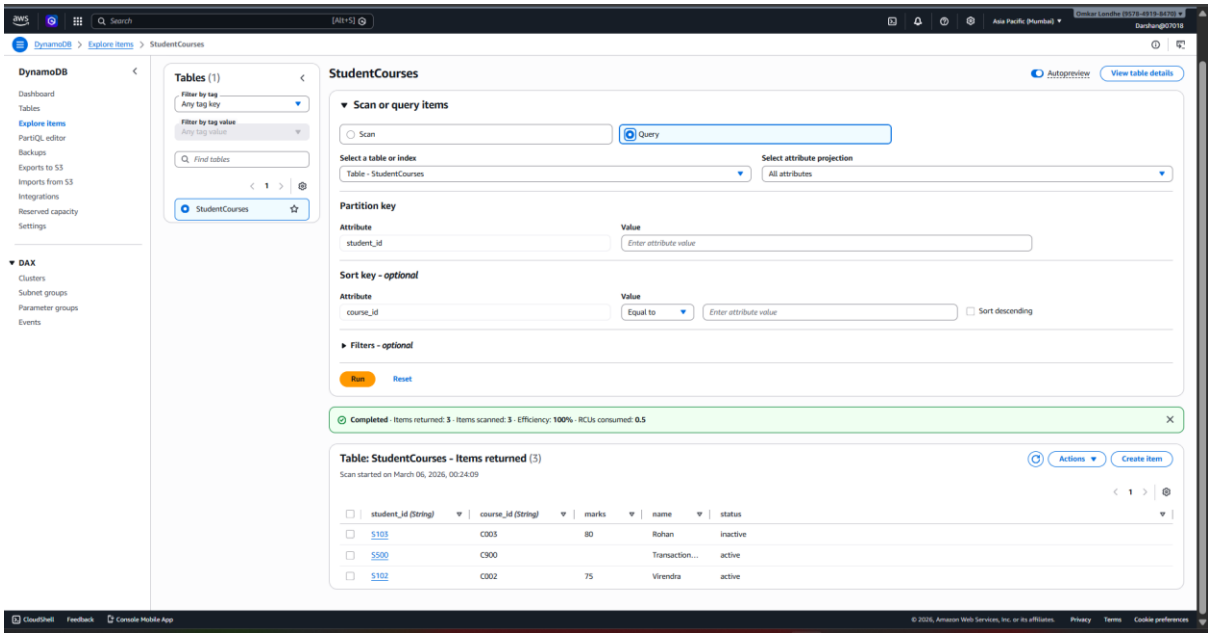
```
(.venv) PS D:\TY\Cloud\CNA term 6> & "d:\TY\Cloud\CNA term 6\.venv\bin\python.exe" "d:/TY/Cloud/CNA term 6/Practical_4_dynamodb/main.py"
Table created successfully
Item inserted
{'student_id': 'S101', 'name': 'Darshan', 'marks': Decimal('85'), 'status': 'active', 'course_id': 'C001'}
Marks updated
Batch insert completed
Transaction completed
Item deleted successfully
```




DynamoDB > Tables > StudentCourses
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Estimated read/write capacity cost
Auto scaling activities (0)
Recent events of automatic scaling. Learn more
Start time
End time
Target
Capacity unit
Description
Status
No auto scaling activities found
There are no auto scaling activities for the table or its global secondary indexes.
Warm throughput Info
Prepare your table for planned peak events, without impacting your application performance or availability. Learn more about Amazon DynamoDB pricing
Edit
Name
Status
Type
Read units per second
Write operations per second
StudentCourses
Active
Table
5
5
StatusIndex
Active
Index
5
5
Deletion protection Info
Protects the table from being deleted unintentionally. When this setting is on, you can't delete the table.
Turn on
Deletion protection
Off
Time to Live (TTL) Info
Automatically delete expired items from a table.
Run preview
Turn on
TTL status
Off
Encryption Info
Provides enhanced security by encrypting all your data at rest using encryption keys stored in AWS Key Management Service.
Manage encryption
Key management
AWS owned key


DynamoDB > Tables > StudentCourses
Share your feedback on Amazon DynamoDB
Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.
Share feedback
Tables (1)
Filter by tag
Any tag key
Filter by tag value
Any tag value
Find tables
StudentCourses
StudentCourses
Actions
Explore table items
Settings
Indexes
Monitor
Global tables
Backups
Exports and streams
Permissions
Global secondary indexes (1) Info
Delete
Create index
Find indexes
Name
Status
Partition key
Sort key
Read capacity
Write capacity
Projected attributes
Size
Item count
StatusIndex
Active
status (String)
-
5
Auto scaling is off
5
Auto scaling is off
All
0 bytes
0


DynamoDB > Explore items > StudentCourses
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Share your feedback on Amazon DynamoDB
Your feedback is an important part of helping us provide a better customer experience. Take this short survey to let us know how we're doing.
Share feedback
Tables (1)
Filter by tag
Any tag key
Filter by tag value
Any tag value
Find tables
StudentCourses
StudentCourses
Autopreview
View table details
Scan or query items
Scan
Query
Select a table or index
Table - StudentCourses
Select attribute projection
All attributes
Filters - optional
Run
Reset
Completed - Items returned: 3 - Items scanned: 3 - Efficiency: 100% - RCUs consumed: 0.5
Table: StudentCourses - Items returned (3)
Scan started on March 06, 2026, 00:24:09
Actions
Create item
student_id (String)
course_id (String)
marks
name
status
S103
C003
80
Rohan
inactive
S500
C900
Transaction...
active
S102
C002
75
Virendra
active


DynamoDB > Explore items > StudentCourses
DynamoDB
Dashboard
Tables
Explore items
PartiQL editor
Backups
Exports to S3
Imports from S3
Integrations
Reserved capacity
Settings
DAX
Clusters
Subnet groups
Parameter groups
Events
Tables (1)
Filter by tag
Any tag key
Filter by tag value
Any tag value
Find tables
StudentCourses
StudentCourses
Autopreview
View table details
Scan or query items
Scan
Query
Select a table or index
Table - StudentCourses
Select attribute projection
All attributes
Partition key
Attribute
student_id
Value
Enter attribute value
Sort key - optional
Attribute
course_id
Value
Equal to
Enter attribute value
Sort descending
Filters - optional
Run
Reset
Completed - Items returned: 3 - Items scanned: 3 - Efficiency: 100% - RCUs consumed: 0.5
Table: StudentCourses - Items returned (3)
Scan started on March 06, 2026, 00:24:09
Actions
Create item
student_id (String)
course_id (String)
marks
name
status
S103
C003
80
Rohan
inactive
S500
C900
Transaction...
active
S102
C002
75
Virendra
active
CloudShell
Feedback
Console Mobile App
Privacy
Terms
Cookie preferences